

// Day 47 of 60

DATABASES

Database Indexing (B-Trees)

→ Swipe through all 6 slides



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

The Full Table Scan

Why your query takes 4 seconds

```
SELECT * FROM users  
WHERE age = 30;
```

Without an index:

- DB reads ALL 10M rows
- checks each one
- returns matches

```
// Time complexity:  $O(n)$   
// 10M rows = ~4 seconds
```



Full scans are linear time



What Is an Index?

A separate, organized structure

```
CREATE INDEX idx_age  
ON users(age);
```

What happens:

DB builds a B-Tree
of (age → row_pointer)
separate from the table

```
// Storage cost: ~10-15%  
// Query speedup: 1000x+
```



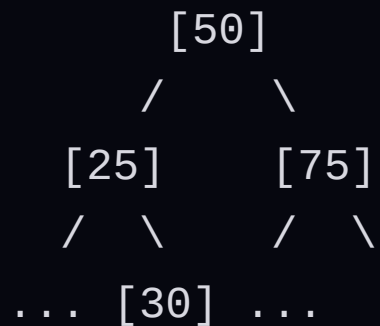
Indexes are duplicated, sorted data



How a B-Tree Works

Sorted, balanced, traversable

Looking for age = 30:



3 hops to find it
out of 10M rows

// Tree is always balanced



B = Balanced (always re-balanced)



Logarithmic Time

Why scale doesn't hurt

B-Tree lookup: $O(\log n)$

1,000 rows → ~10 hops

1,000,000 rows → ~20 hops

1,000,000,000 → ~30 hops

// 1B rows = 30 disk reads

// vs 1B reads (full scan)



Logarithmic curves are nearly flat at scale



The Dark Side

Every index taxes your writes

On INSERT / UPDATE:

1. Update the table
2. Update EVERY index
3. Rebalance B-Trees

5 indexes = 5x write cost

// More indexes

// = faster reads

// = slower writes

// = bigger storage



Audit unused indexes regularly



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

The Golden Rule

Don't index every column

Index the columns used in:

- WHERE clauses
- JOIN conditions
- ORDER BY clauses

Don't index:

- Low-cardinality columns
(gender, bool flags)
- Rarely queried columns
- Columns that change a lot



Cardinality matters more than column count



done = true

Follow for more!

Like · Comment · Repost

Shubham Bhati

linkedin.com/in/bhatishubham



Shubham Bhati

linkedin.com/in/bhatishubham